Chapter 1

Introduction to FoodBridge Backend

A Spring Boot backend connecting food donors with NGOs to reduce food wastage — covering architecture, request flow, and the authentication module.

1.1 Project Overview

FoodBridge is a Spring Boot backend application whose primary objective is to reduce food wastage by connecting food donors with NGOs. Instead of allowing excess food to be discarded, the application provides a platform where donors can register available food and NGOs can claim it before it goes to waste.

The backend is responsible for handling all the business logic of the application. It receives requests from the client, validates them, processes the required operations, communicates with the database, and finally returns an appropriate response.

At the beginning of the project, our primary goal is to build a secure authentication system. Before implementing food donation or NGO management, we must ensure that only legitimate users can access the application's protected resources.

1.2 Functionalities Implemented So Far

Till now, the backend supports the following functionalities, which together form the authentication module of our application:

- User Registration
- User Login
- Password Encryption using BCrypt
- JWT Generation
- JWT Authentication
- Spring Security Configuration
- Protection of Private APIs

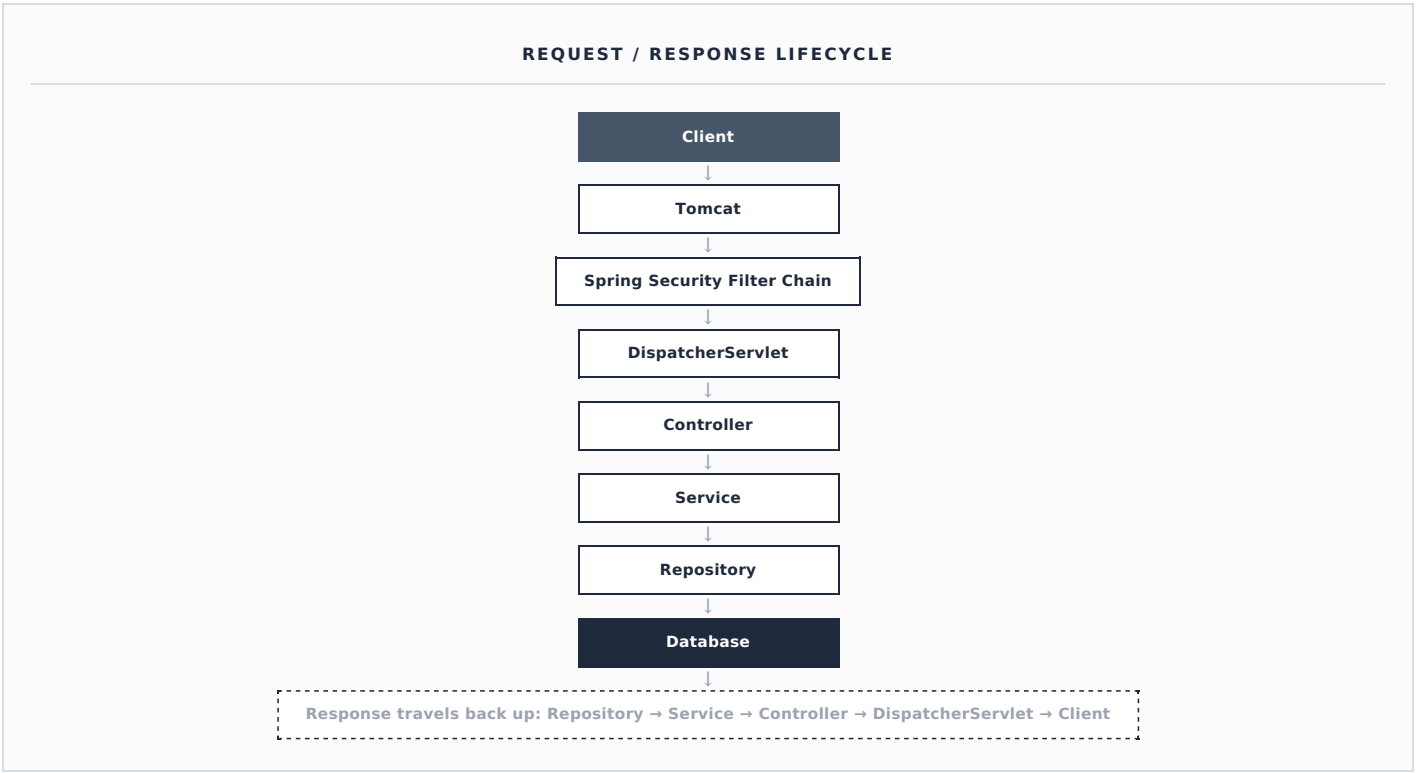
1.3 Technology Stack

The backend is built using the following technologies, each with a specific responsibility:

TECHNOLOGY	RESPONSIBILITY
Java	Provides the core programming language
Spring Boot	Simplifies development with auto-configuration and an embedded web server
Spring MVC	Handles incoming HTTP requests
Spring Security	Provides authentication and authorization
Spring Data JPA	Simplifies database operations
Hibernate	Acts as the ORM layer between Java objects and database tables
MySQL	Stores the application's persistent data
Maven	Manages project build and dependencies
JWT (JSON Web Token)	Provides stateless authentication

1.4 Backend Architecture

Our application follows a **Layered Architecture**. Every request follows the same path down through the layers and back up again:



Each layer has only one responsibility: the **Controller** receives HTTP requests, the **Service** contains business logic, and the **Repository** communicates with the database. This separation makes the application easier to understand, maintain, and test.

1.5 Why Layered Architecture?

Imagine writing every piece of code inside a single class — the controller receiving the request, validating data, performing business logic, executing SQL queries, generating responses, and handling exceptions all at once. Such code quickly becomes difficult to maintain.

Instead, Spring encourages dividing responsibilities among multiple layers, where each layer performs only one task. This principle is known as the **Separation of Concerns**. Because every class has only one responsibility, modifying one part of the application has minimal impact on the others.

1.6 How a Request Travels Through the Application

Whenever the client sends an HTTP request, it first reaches Tomcat, which accepts the request and creates two important objects — **HttpServletRequest** and **HttpServletResponse** — that are passed throughout the entire request lifecycle.

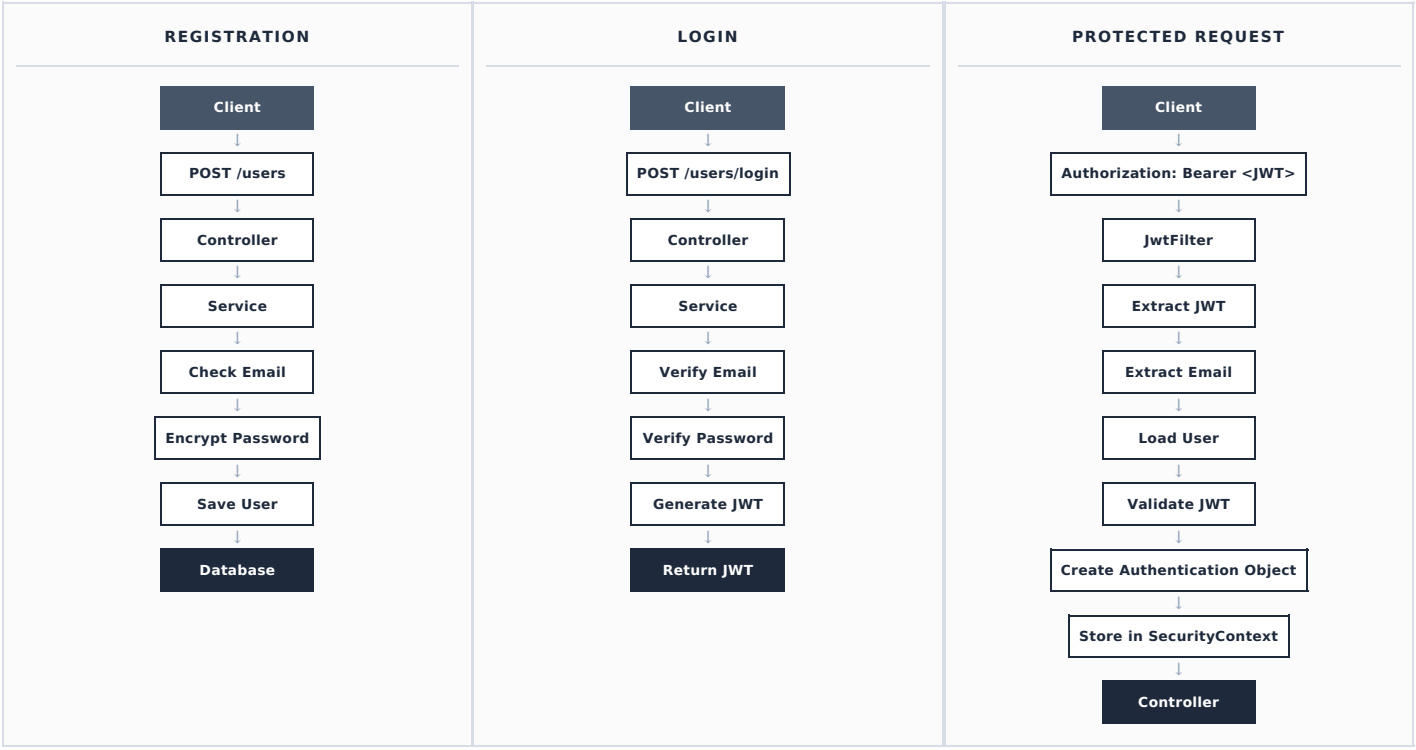
The request then enters the **Spring Security Filter Chain**, which decides whether the request should be allowed to proceed. If authentication is required, the security filters perform the necessary authentication checks.

Once security verification is complete, the request reaches the **DispatcherServlet**, which acts as the front controller of Spring MVC and identifies the correct controller method for the incoming request.

The controller does not contain business logic — instead, it delegates work to the **Service** layer, which performs validations, applies business rules, and requests database operations from the **Repository**. The Repository communicates with MySQL through Spring Data JPA and Hibernate. After the database operation completes, the response travels back through the same layers until it finally reaches the client.

1.7 Authentication Module (Current Progress)

The authentication module currently implements three key flows, illustrated below.



1.8 What We Have Learned

After completing the authentication module, we now understand the following concepts, which form the foundation for the remaining modules of the FoodBridge project:

Layered Architecture	**Spring Boot Request Flow**	**Controller Layer**	**Service Layer**	**Repository Layer**	**Password Encryption**
BCryptPasswordEncoder	**JWT Authentication**	**Spring Security**	**OncePerRequestFilter**	**SecurityFilterChain**	**JwtFilter**	**JwtService**
Authentication	**UsernamePasswordAuthenticationToken**	**SecurityContext**	**SecurityContextHolder**		

CHAPTER SUMMARY

At this stage, we have successfully built a secure authentication system for FoodBridge. Users can register with encrypted passwords, log in using their credentials, receive a signed JWT upon successful authentication, and use that JWT to access protected endpoints. Spring Security validates the token before allowing any protected request to reach the application's business logic.

> *The next chapter will explain, in detail, how Spring Boot starts internally — from executing the `main()` method to creating beans, starting Tomcat, and preparing the application to handle incoming requests.*